SESSION 13

Assessment Submission

Accessibility Audit, Superheroes Feature & 30-Day Compose Roadmap

Student: Sudarshan G

Course: Android App Development with Kotlin & Jetpack Compose

Session: 13 — Accessibility, Large-Scale Features & Project Planning

Tasks: Task 1 · Task 2 · Task 3

TASK 1 — Accessibility Audit + Improve Plan

1. UI Screen Selected

Screen audited: WanderApp Travel Destination Card Gallery (Session 12, Task 3/4)

The screen selected for the accessibility audit is the Travel Destination Card Gallery built in Session 12 — the primary browsing screen of WanderApp. It includes a TopAppBar, a filter chip row, a LazyColumn of destination cards (each with an AsyncImage, city name, country text, rating, cost chip, favourite IconButton, and an expand/collapse interaction), and a NavigationBar. This screen was chosen because it contains the widest variety of interactive elements — images, touch targets, animated icons, chips, and dynamic content — making it the most meaningful screen to audit for accessibility coverage. The audit covers the screen in both its collapsed and expanded card states, and with dark mode active.

2. Accessibility Audit Checklist

#	Audit Area	Check Item	Status	Evidence / Notes
1	contentDescription — meaningful images	AsyncImage hero photos have contentDescription set to '${destination.cityName} photo'	PASS	Code: contentDescription = '${dest.cityName}, ${dest.country} travel photo'. TalkBack reads it correctly.
2	contentDescription — decorative images	Decorative placeholder/shimmer images have contentDescription = null	PASS	Placeholder shimmer uses null contentDescription so TalkBack skips it during loading.

#	Audit Area	Check Item	Status	Evidence / Notes
3	contentDescription — favourite icon	Heart icon contentDescription updates on state change: 'Add to favourites' vs 'Remove from favourites'	NEED IMPROVEMENT	Found: static contentDescription 'Favourite' regardless of state. Fix: conditionally set based on isFavourite.
4	contentDescription — cost tier chip	FilterChip for cost tier has readable label not just icon	PASS	Chip label text 'Budget', 'Mid-Range', 'Luxury' is used — no icon-only chip.
5	Touch target — favourite icon	IconButton for favourite is at least 48×48dp	PASS	IconButton default minimum size is 48×48dp per Material 3 spec. No override applied.
6	Touch target — filter chips	Filter chips have height ≥ 48dp	NEED IMPROVEMENT	FilterChip default height is 32dp. Need to add Modifier.heightIn(min = 48.dp) or padding wrapper.
7	Touch target — card tap area	Card tap area (expand) covers full card width	PASS	Modifier.fillMaxWidth().clickable covers full card width. Compliant.
8	Touch target — navigation bar items	NavigationBar items are at least 48×48dp	PASS	Material 3 NavigationBar items meet 48dp minimum by default.
9	Colour contrast — city name (light)	onSurface (#1A1A2E) on surface (#FFFFFF): contrast ratio	PASS	Contrast ratio: 16.1:1. Exceeds WCAG AAA (7:1). ✓
10	Colour contrast — body text (light)	onSurfaceVariant on surfaceVariant expanded card	PASS	Calculated ratio ~5.8:1. Meets WCAG AA (4.5:1). ✓
11	Colour contrast — filter chip label (light)	onSecondaryContainer on secondaryContainer	PASS	Ratio ~6.2:1. Meets WCAG AA. ✓
12	Colour contrast — dark mode city name	onSurface (#E8EAF6) on surface (#1A1A2E): contrast ratio	PASS	Ratio: 14.7:1. Exceeds AAA. ✓
13	Colour contrast — grey secondary text	Gray (#555555) on white (#FFFFFF)	NEED IMPROVEMENT	Ratio 7.0:1 passes AA, but used for rating text at 12sp (small text). For text below 14sp, AAA (7:1) recommended. Currently borderline.
14	Motion reduction — card expand	animateContentSize + updateTransition respect animatorDurationScale = 0	NEED IMPROVEMENT	Not currently checked. Need to add LocalAccessibilityManager / animatorDurationScale check and use snap() when scale = 0.
15	Motion reduction — favourite icon	keyframes scale animation respects Reduce Motion	NEED IMPROVEMENT	Same as above — no animatorDurationScale guard implemented.

#	Audit Area	Check Item	Status	Evidence / Notes
16	Motion reduction — list item entry	AnimatedVisibility fadeIn/slideIn respects Reduce Motion	NEED IMPROVEMENT	AnimatedVisibility enter/exit specs not conditioned on motion preference.
17	Focus order — card	TalkBack focus moves: image → title → country → rating → cost → favourite icon	PASS	Compose default focus order follows layout order. Verified with TalkBack.
18	Focus order — expanded card	TalkBack includes description + 'View Details' button in focus sequence	PASS	Conditional content added in-tree order — accessible in order when expanded.
19	Focus order — filter chips	Filter chip row is reachable and announces correctly	NEED IMPROVEMENT	LazyRow of chips lacks a group semantics label. TalkBack jumps into chips without context. Add Modifier.semantics { contentDescription = 'Filter options' } to the Row.
20	Text scalability — city name	titleLarge (22sp) wraps correctly at 200% font scale	PASS	Text uses .wrapContentHeight() on Column, no fixed heights — wraps at large font.
21	Text scalability — filter chip labels	Chip labels wrap or scroll at 200% font scale	NEED IMPROVEMENT	Chip labels in LazyRow truncate at large font scale. Need maxLines=1 with ellipsis or sp-based min width.
22	Semantic role — card clickable	Card clickable has semantic role Role.Button or meaningful stateDescription	NEED IMPROVEMENT	Modifier.clickable on Card has no semantics block. TalkBack says 'Double tap to activate' with no context. Add: Modifier.semantics { stateDescription = if (isExpanded) 'Expanded' else 'Collapsed'; role = Role.Button }.
23	Screen reader — search field	SearchBar/OutlinedTextField has label readable by TalkBack	PASS	label = { Text('Search destinations') } is present. TalkBack reads 'Search destinations, edit text'.

3. Improvement Plan Document

#	Issue	Concrete Fix	Priority	Stage
A3	Static favourite contentDescription	Change to: contentDescription = if (dest.isFavourite) 'Remove ${dest.cityName} from favourites' else 'Add ${dest.cityName} to favourites'	HIGH	Stage 1 — immediate (1 line change per card)
A6	Filter chip touch target < 48dp	Wrap FilterChip in Box with Modifier.heightIn(min = 48.dp).clickable { } or add padding: Modifier.padding(vertical = 8.dp)	HIGH	Stage 1 — immediate (add modifier)
A14–16	Animations do not respect Reduce Motion	Add: val animScale = Settings.Global.getFloat(context.contentResolver, Settings.Global.ANIMATOR_DURATION_SCALE, 1f); replace animationSpec with if (animScale == 0f) snap() else springSpec everywhere	HIGH	Stage 2 — requires refactor of all animation specs to accept conditional spec
A19	Filter chip row lacks group semantics label	Add to LazyRow wrapper: Modifier.semantics(mergeDescendants = false) { contentDescription = 'Filter destination list' }	MEDIUM	Stage 1 — single modifier addition
A21	Chip labels truncate at large font scale	Set label text with overflow = TextOverflow.Ellipsis, maxLines = 1; or set Modifier.wrapContentWidth() on chip for scrollable LazyRow	MEDIUM	Stage 2 — test at 200% scale after fix
A22	Card clickable lacks semantic context	Add: Modifier.semantics { stateDescription = if (isExpanded) 'Expanded, shows full description' else 'Collapsed, tap to expand'; role = Role.Button }	HIGH	Stage 1 — add to each DestinationCard
A13	Rating text at 12sp borderline contrast	Increase rating text to 14sp (bodySmall → bodyMedium) or use colorScheme.onSurface instead of onSurfaceVariant for rating row	LOW	Stage 3 — design review required before change

3.1 Implementation Stages

Stage	Description	Issues Fixed	Est. Time
Stage 1	Quick wins — single-line modifier and label fixes	A3, A6, A19, A22	2 hours
Stage 2	Reduce Motion refactor — create animSpec() helper	A14, A15, A16, A21	3 hours
Stage 3	Design review — contrast and font size decisions	A13	1 hour + designer sign-off

3.2 Reduce Motion Helper Implementation

```
// AccessibilityUtils.kt
@Composable
```

```
fun rememberAnimationSpec(): Boolean {
    val context = LocalContext.current
    return remember {
        Settings.Global.getFloat(
            context.contentResolver,
            Settings.Global.ANIMATOR_DURATION_SCALE,
            1f
        ) > 0f  // false when user has disabled animations
    }
}

// Usage in composable:
val animEnabled = rememberAnimationSpec()

val cardExpandSpec = if (animEnabled)
    spring(dampingRatio = Spring.DampingRatioMediumBouncy, stiffness =
Spring.StiffnessLow)
else
    snap()

val heartScaleSpec = if (animEnabled)
    keyframes { durationMillis = 300; 1.3f at 150; 1.0f at 300 }
else
    snap()

// All animate*AsState calls now use these conditional specs
val heartScale by animateFloatAsState(
    targetValue = if (isFavourite) 1.0f else 1.0f,
    animationSpec = heartScaleSpec,
    label = "heart_scale"
)

// AnimatedVisibility also conditioned:
AnimatedVisibility(
    visible = true,
    enter = if (animEnabled) fadeIn(tween(300)) + slideInVertically() else
EnterTransition.None,
    exit  = if (animEnabled) fadeOut(tween(200)) else ExitTransition.None
)
```

4. Integration with Automated Tests

4.1 Proposed Automated Accessibility Test Cases

```
// AccessibilityTest.kt — using Compose UI Test + Semantics
@get:Rule
val composeTestRule = createComposeRule()

// ── Test 1: Hero photo has non-empty contentDescription ──────────
@Test
fun heroPhoto_hasContentDescription() {
    composeTestRule.setContent {
        WanderAppTheme { DestinationCard(sampleDestinations.first(), false, {}, {}) }
```

```
    }
    composeTestRule
        .onNode(hasContentDescription("Kyoto, Japan travel photo"))
        .assertIsDisplayed()
}

// ── Test 2: Favourite icon contentDescription changes on toggle ───
@Test
fun favouriteIcon_contentDescriptionUpdatesOnToggle() {
    var isFav = false
    composeTestRule.setContent {
        WanderAppTheme {
            DestinationCard(
                destination = sampleDestinations.first().copy(isFavourite = isFav),
                isExpanded = false,
                onFavToggle = { isFav = !isFav },
                onExpandToggle = {}
            )
        }
    }
    // Initial state: not favourited
    composeTestRule
        .onNode(hasContentDescription("Add Kyoto to favourites"))
        .assertIsDisplayed()
    // After tap
    composeTestRule
        .onNode(hasContentDescription("Add Kyoto to favourites"))
        .performClick()
    composeTestRule
        .onNode(hasContentDescription("Remove Kyoto from favourites"))
        .assertIsDisplayed()
}

// ── Test 3: Filter chip touch target meets 48dp minimum ──────────
@Test
fun filterChip_touchTargetMeetsMinimumSize() {
    composeTestRule.setContent {
        WanderAppTheme { DestinationFilterChipRow(onFilterChange = {}) }
    }
    composeTestRule
        .onNode(hasText("All"))
        .assertHeightIsAtLleast(48.dp)
}

// ── Test 4: Expanded card semantic state description ──────────────
@Test
fun card_semanticStateDescriptionUpdatesOnExpand() {
    var expanded = false
    composeTestRule.setContent {
        WanderAppTheme {
            DestinationCard(
                destination = sampleDestinations.first(),
                isExpanded = expanded,
                onExpandToggle = { expanded = !expanded },
                onFavToggle = {}
```

```
            )
        }
    }
    composeTestRule
        .onNode(hasStateDescription("Collapsed, tap to expand"))
        .assertIsDisplayed()
    composeTestRule
        .onNode(hasStateDescription("Collapsed, tap to expand"))
        .performClick()
    composeTestRule
        .onNode(hasStateDescription("Expanded, shows full description"))
        .assertIsDisplayed()
}

// ── Test 5: No animation runs when ANIMATOR_DURATION_SCALE = 0 ───
@Test
fun reduceMotion_animationsSkippedWhenScaleZero() {
    // Set animator scale to 0 programmatically in test
    Settings.Global.putFloat(
        InstrumentationRegistry.getInstrumentation().targetContext.contentResolver,
        Settings.Global.ANIMATOR_DURATION_SCALE, 0f
    )
    composeTestRule.setContent {
        WanderAppTheme { DestinationCard(sampleDestinations.first(), false, {}, {}) }
    }
    // Expand card — content should appear immediately (no animation delay)
    composeTestRule
        .onNode(hasStateDescription("Collapsed, tap to expand"))
        .performClick()
    // With snap(), content is immediately present — no 300ms wait needed
    composeTestRule
        .onNode(hasText(sampleDestinations.first().description))
        .assertIsDisplayed()
}

// ── Test 6: All images on list screen have non-null contentDescription
@Test
fun allHeroImages_haveNonNullContentDescription() {
    composeTestRule.setContent {
        WanderAppTheme {
            DestinationListScreen(viewModel = testViewModel())
        }
    }
    // Check that no Image node in the tree has a null or empty contentDescription
    composeTestRule
        .onAllNodes(hasContentDescriptionExactly(""))  // should find ZERO
        .assertCountEquals(0)
}
```

4.2 Testing Libraries & APIs Used

Library / API	Purpose	Dependency
androidx.compose.ui:ui-test-junit4	composeTestRule, setContent, onNode	androidTestImplementation
SemanticsMatcher (hasContentDescription, hasStateDescription, hasText)	Find nodes by semantic properties	Included in ui-test
Modifier.semantics { stateDescription, role }	Declare semantic state and role in production code	Core Compose — no extra dep
Modifier.testTag + hasTestTag	Find composables by tag in tests	Included in ui-test
assertHeightIsAtLeast(48.dp)	Verify touch target size programmatically	ui-test SemanticsNodeInteraction
AccessibilityChecks (Espresso)	Run automated WCAG contrast + target size checks	com.google.android.apps.common.testing.accessibility.framework
InstrumentationRegistry	Set system settings (animatorDurationScale) in tests	androidx.test:runner

5. Reflection

Accessibility is not an optional feature or a late-stage polish step — it is a fundamental quality criterion that determines whether an app is usable by its full potential audience. In India alone, over 70 million people have some form of disability, and globally over 1 billion people rely on assistive technologies to use digital products. An app that fails basic accessibility standards — missing content descriptions, insufficient touch targets, animations that cause vestibular discomfort — actively excludes a significant portion of potential users. Beyond inclusiveness, accessibility compliance is increasingly a legal requirement (WCAG 2.1 AA is the baseline in many jurisdictions), and many enterprise clients mandate it as a procurement criterion. Planning and testing for accessibility from the start — before a line of UI code is written — improves quality in two ways: it forces clearer semantic thinking (every interactive element must have a purpose and a label), and it catches costly architectural problems early. The most significant challenge discovered during this audit was the Reduce Motion issue: adding a global animatorDurationScale guard required refactoring every animate* call in the codebase to accept a conditional spec, because the specs had been hard-coded throughout. Had the rememberAnimationSpec() helper been created on Day 1 and used consistently, this would have been a single decision point rather than a refactoring task. This reinforced the principle from Session 12 — build accessibility into the architecture before building features, not after.

TASK 2 — Large-Scale UI Feature: Superheroes List with Tests & Accessibility

1. Feature Description

The Superheroes Directory is a full-scale, multi-screen Android app feature that provides a browsable, searchable, and filterable catalogue of 50+ superheroes from the Marvel and DC universes. Each hero is presented with their photo, real name, alias, a power summary, and a favourite toggle. Users can search by hero name or power keyword in real time, filter the list to show only favourited heroes, and tap any hero card to navigate to a dedicated detail screen showing their full profile. The feature is built with accessibility from day one — every interactive element has semantic labels, touch targets meet 48dp minimums, and all animations respect the Reduce Motion setting — and is covered by a comprehensive suite of automated UI and unit tests.

2. Data Model & Dataset Planning

2.1 Superhero Data Class

```
data class Superhero(
    val id: Int,
    val name: String,                    // Real name (e.g., 'Peter Parker')
    val heroName: String,                // Hero alias (e.g., 'Spider-Man')
    val alias: String?,                  // Additional codename if any (nullable)
    val powerSummary: String,            // 2-3 sentence power description
    val powerKeywords: List<String>,     // For search/filter (e.g., ['strength',
'flight'])
    val universe: String,                // 'Marvel' or 'DC'
    val imageUrl: String,                // Remote URL via Coil
    val imageResId: Int,                 // Local drawable fallback
    val isFavourite: Boolean = false,
    val firstAppearance: String = "",   // e.g. '1962'
    val team: String? = null             // e.g. 'Avengers', nullable
)

// Extension for display
fun Superhero.displayName(): String =
    if (alias != null) "$heroName / $alias" else heroName
```

2.2 Sample Dataset — 12 Heroes

ID	Real Name	Hero Name	Universe	Power Keywords	Team	Image Resource
1	Peter Parker	Spider-Man	Marvel	wall-crawl, web, agility, spider-sense	Avengers	hero_spiderman
2	Tony Stark	Iron Man	Marvel	tech, flight, strength, genius, armour	Avengers	hero_ironman

ID	Real Name	Hero Name	Universe	Power Keywords	Team	Image Resource
3	Diana Prince	Wonder Woman	DC	strength, flight, lasso, warrior, demigod	Justice League	hero_wonderwoman
4	Bruce Wayne	Batman	DC	detective, martial arts, tech, stealth, no powers	Justice League	hero_batman
5	Natasha Romanoff	Black Widow	Marvel	espionage, martial arts, stealth, agility	Avengers	hero_blackwidow
6	T'Challa	Black Panther	Marvel	vibranium, strength, agility, tech, king	Avengers	hero_blackpanther
7	Clark Kent	Superman	DC	flight, strength, heat vision, invulnerability	Justice League	hero_superman
8	Steve Rogers	Captain America	Marvel	super soldier, shield, strength, leadership	Avengers	hero_captainamerica
9	Barry Allen	The Flash	DC	speed, time travel, lightning, reflexes	Justice League	hero_flash
10	Wanda Maximoff	Scarlet Witch	Marvel	magic, reality warp, telekinesis, mind	Avengers	hero_scarletwitch
11	Arthur Curry	Aquaman	DC	underwater, strength, trident, telepathy	Justice League	hero_aquaman
12	Thor Odinson	Thor	Marvel	lightning, Mjolnir, strength, flight, god	Avengers	hero_thor

```
// Sample dataset (first 3 shown, 12 total in full implementation)
val sampleHeroes = listOf(
    Superhero(
        id = 1, name = "Peter Parker", heroName = "Spider-Man", alias = "Web-Slinger",
        powerSummary = "Bitten by a radioactive spider, Peter gained superhuman agility,"
+
                        " wall-crawling, and a sixth sense. His web-shooters let him" +
                        " swing across New York and neutralise enemies.",
        powerKeywords = listOf("wall-crawl", "web", "agility", "spider-sense"),
        universe = "Marvel", imageUrl = "https://cdn.superheroapi.com/spiderman.jpg",
        imageResId = R.drawable.hero_spiderman, isFavourite = true,
        firstAppearance = "1962", team = "Avengers"
    ),
    Superhero(
        id = 2, name = "Tony Stark", heroName = "Iron Man", alias = "Shellhead",
```

```
        powerSummary = "Genius billionaire Tony Stark built a powered armour suit to
escape" +
                        " captivity, then upgraded it to become one of Earth's mightiest"
+
                        " defenders. His suits provide flight, strength, and advanced
weapons.",
        powerKeywords = listOf("tech", "flight", "strength", "genius", "armour"),
        universe = "Marvel", imageUrl = "https://cdn.superheroapi.com/ironman.jpg",
        imageResId = R.drawable.hero_ironman, isFavourite = false,
        firstAppearance = "1963", team = "Avengers"
    ),
    Superhero(
        id = 3, name = "Diana Prince", heroName = "Wonder Woman", alias = null,
        powerSummary = "Daughter of Zeus and the Amazonian queen, Diana wields divine" +
                        " strength, flight, and the Lasso of Truth. A warrior princess" +
                        " who serves as humanity's greatest protector.",
        powerKeywords = listOf("strength", "flight", "lasso", "warrior", "demigod"),
        universe = "DC", imageUrl = "https://cdn.superheroapi.com/wonderwoman.jpg",
        imageResId = R.drawable.hero_wonderwoman, isFavourite = true,
        firstAppearance = "1941", team = "Justice League"
    ),
    // ... 9 more heroes
)
```

3. UI Layout Plans

3.1 Heroes List Screen

UI Zone	Composable	Data / State	Theme Token	Accessibility
TopAppBar	TopAppBar	Title: 'Heroes Directory'	colorScheme.primary + typography.titleLarge	N/A — static title
Search bar	SearchBar / OutlinedTextField	searchQuery: String (ViewModel)	colorScheme.surface, shapes.small	label='Search heroes by name or power'; contentDescription set
Filter row — Universe	FilterChip (Marvel/DC/All)	selectedUniverse: String (ViewModel)	colorScheme.secondaryContainer when selected	Chip labels readable; heightIn(min=48.dp)
Filter row — Favourites	FilterChip (All/Favourites)	filterFavourite: Boolean (ViewModel)	selectedContainerColor = secondaryContainer	Label 'Show favourites only'; role = Role.Checkbox
Hero list	LazyColumn (portrait) / LazyVerticalGrid(2) (landscape)	displayedHeroes: List<Superhero>	Card: shapes.medium, surface	key = { it.id }; AnimatedVisibility per item

UI Zone	Composable	Data / State	Theme Token	Accessibility
HeroCard — image	AsyncImage + Box overlay	hero.imageUrl + imageResId fallback	Clip: shapes.medium top corners	contentDescription = '${hero.heroName} photo'
HeroCard — name	Text titleLarge	hero.heroName	typography.titleLarge, onSurface	N/A — read by TalkBack in order
HeroCard — real name	Text bodySmall	hero.name	typography.bodySmall, onSurfaceVariant	N/A — read by TalkBack
HeroCard — power	Text bodySmall, maxLines=2	hero.powerSummary (truncated)	typography.bodySmall	N/A — read by TalkBack
HeroCard — favourite icon	IconButton + Icon	hero.isFavourite (ViewModel)	colorScheme.error when active	contentDescription = 'Add/Remove ${hero.heroName} from favourites'
HeroCard — universe badge	AssistChip	hero.universe (Marvel/DC)	tertiary/secondary chip tonal	Label 'Marvel' or 'DC' readable
NavigationBar	NavigationBar with 3 items	current route	colorScheme.surfaceContainer	NavigationBarItem labels present; indicatorColor

3.2 Hero Detail Screen

UI Zone	Composable	Data	Theme Token	Accessibility
Back navigation	TopAppBar with navigationIcon (ArrowBack)	Back to list	colorScheme.primary	contentDescription = 'Back to heroes list'; role = Role.Button
Hero hero image	AsyncImage (full width, 260dp)	hero.imageUrl	contentScale = Crop; clip shapes.large bottom	contentDescription = '${hero.heroName} full portrait'
Hero name + alias	Row: Text (displayLarge) + Chip (alias)	hero.heroName + hero.alias?	typography.displayLarge	alias chip has label contentDescription
Real name row	Row: Icon + Text	hero.name	typography.titleMedium, onSurfaceVariant	Icon: contentDescription = 'Real name'; decorative=false
Universe + Team badges	Row: AssistChip + AssistChip	hero.universe + hero.team?	tertiary chip, secondary chip	Chip labels readable by TalkBack
First appearance	Row: Icon + Text	hero.firstAppearance	typography.bodyMedium	Icon: contentDescription

UI Zone	Composable	Data	Theme Token	Accessibility
				= 'First appeared in'
Power summary	Text headlineMedium + Text bodyMedium	hero.powerSummary	typography.headlineMedium for section header	Section header has semantics heading = true
Power keyword chips	LazyRow of FilterChips	hero.powerKeywords	secondaryContainer chips	Row semantics label = 'Power keywords'
Favourite button	FilledButton (large, full width)	hero.isFavourite (ViewModel)	colorScheme.primary / secondary container	contentDescription changes with state; role = Role.Button

4. State & Logic Flow

4.1 State Variables

```
class SuperheroViewModel : ViewModel() {

    // ── Master data ──────────────────────────────────────────────
    private val _allHeroes = MutableStateFlow(sampleHeroes)
    val allHeroes = _allHeroes.asStateFlow()

    // ── Filter state ─────────────────────────────────────────────
    private val _searchQuery = MutableStateFlow("")
    val searchQuery = _searchQuery.asStateFlow()

    private val _filterFavourite = MutableStateFlow(false)
    val filterFavourite = _filterFavourite.asStateFlow()

    private val _selectedUniverse = MutableStateFlow("All")  // 'All', 'Marvel', 'DC'
    val selectedUniverse = _selectedUniverse.asStateFlow()

    private val _selectedHeroId = MutableStateFlow<Int?>(null)
    val selectedHeroId = _selectedHeroId.asStateFlow()

    // ── Derived displayed list ───────────────────────────────────
    val displayedHeroes: StateFlow<List<Superhero>> =
        combine(_allHeroes, _searchQuery, _filterFavourite, _selectedUniverse) {
            heroes, query, favOnly, universe ->
            heroes.filter { hero ->
                val matchesQuery = query.isBlank() ||
                    hero.heroName.contains(query, ignoreCase = true) ||
                    hero.name.contains(query, ignoreCase = true) ||
                    hero.powerKeywords.any { it.contains(query, ignoreCase = true) }
                val matchesFav    = !favOnly || hero.isFavourite
```

```
                val matchesUniverse = universe == "All" || hero.universe == universe
                matchesQuery && matchesFav && matchesUniverse
            }
        }.stateIn(viewModelScope, SharingStarted.Lazily, sampleHeroes)

    // ── Actions ──────────────────────────────────────────────────
    fun updateSearchQuery(query: String) { _searchQuery.value = query }
    fun toggleFavouriteFilter() { _filterFavourite.value = !_filterFavourite.value }
    fun selectUniverse(universe: String) { _selectedUniverse.value = universe }
    fun selectHero(heroId: Int) { _selectedHeroId.value = heroId }

    fun toggleFavourite(heroId: Int) {
        _allHeroes.update { list ->
            list.map { if (it.id == heroId) it.copy(isFavourite = !it.isFavourite) else
it }
        }
    }

    // ── Derived: selected hero ───────────────────────────────────
    val selectedHero = combine(_allHeroes, _selectedHeroId) { heroes, id ->
        heroes.firstOrNull { it.id == id }
    }.stateIn(viewModelScope, SharingStarted.Lazily, null)
}
```

4.2 Filtered List Logic (Expanded)

```
// The combine() operator merges 4 StateFlows into one derived list:
// hero visible if ALL of these conditions are true:
//
// 1. matchesQuery:
//    query is blank (no filter) OR
//    heroName contains query (case-insensitive) OR
//    real name contains query OR
//    any powerKeyword contains query
//
// 2. matchesFav:
//    filterFavourite is false (show all) OR
//    hero.isFavourite is true
//
// 3. matchesUniverse:
//    selectedUniverse == 'All' OR
//    hero.universe == selectedUniverse

// Example results for various filter combinations:
// searchQuery='flight', filterFav=false, universe='All':
//   → Iron Man, Wonder Woman, Superman, Thor (have 'flight' in powerKeywords)

// searchQuery='', filterFav=true, universe='Marvel':
//   → Spider-Man, Wonder Woman excluded (DC), shows only favourited Marvel heroes

// searchQuery='strength', filterFav=false, universe='DC':
//   → Wonder Woman, Superman, Batman? (no 'strength' kw), Aquaman, Flash?
//   → Wonder Woman (#3), Superman (#7), Aquaman (#11)
```

4.3 Favourite Toggle Logic

```
// ViewModel action — immutable update via copy()
```

```
fun toggleFavourite(heroId: Int) {
    _allHeroes.update { list ->
        list.map { hero ->
            if (hero.id == heroId) hero.copy(isFavourite = !hero.isFavourite)
            else hero
        }
    }
    // _allHeroes update triggers displayedHeroes recomputation via combine()
    // -> LazyColumn recomposes affected item only (key = { it.id })
    // -> Favourite icon updates (filled/outlined) and tint changes
    // -> If filterFavourite = true and hero just unfavourited:
    //    hero disappears from list with AnimatedVisibility exit animation
}

// In HeroCard composable:
IconButton(
    onClick = { onFavToggle(hero.id) },
    modifier = Modifier
        .minimumInteractiveComponentSize()  // ensures 48dp
        .semantics {
            contentDescription = if (hero.isFavourite)
                "Remove ${hero.heroName} from favourites"
            else
                "Add ${hero.heroName} to favourites"
        }
) {
    Icon(
        imageVector = if (hero.isFavourite) Icons.Filled.Favorite
                      else Icons.Outlined.FavoriteBorder,
        contentDescription = null,  // set on IconButton, not Icon
        tint = if (hero.isFavourite) MaterialTheme.colorScheme.error
               else MaterialTheme.colorScheme.onSurfaceVariant
    )
}
```

5. Accessibility & Testing Strategy

5.1 Accessibility Features Built In

Element	Accessibility Implementation	Semantic API Used
Hero photo (list)	contentDescription = '${hero.heroName} photo'	AsyncImage contentDescription parameter
Hero photo (detail)	contentDescription = '${hero.heroName} full portrait photo'	AsyncImage contentDescription parameter
Favourite icon	contentDescription = 'Add/Remove ${hero.heroName} from favourites'	Modifier.semantics { contentDescription }
Back navigation icon	contentDescription = 'Navigate back to heroes list'	Icon contentDescription parameter

Element	Accessibility Implementation	Semantic API Used
Search bar	label = 'Search heroes by name or power'	OutlinedTextField label parameter
Filter chip — Favourites	role = Role.Checkbox; stateDescription = 'Favourites only'	Modifier.semantics { role; stateDescription }
Hero card (expand/collapse)	stateDescription = 'Expanded' or 'Collapsed'; role = Role.Button	Modifier.semantics { stateDescription; role }
Power keywords section header	heading = true in semantics	Modifier.semantics { heading() }
Universe badge chip	Chip label text 'Marvel' or 'DC' is readable	AssistChip label parameter
Filter chip row	Group label via semantics on Row	Modifier.semantics { contentDescription = 'Filter options' }

5.2 Automated Test Cases — 8 Tests

```
// SuperheroesTest.kt

// ── Test 1: Search by hero name filters list correctly ──────────
@Test
fun search_byHeroName_filtersListCorrectly() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    composeTestRule
        .onNode(hasText("Search heroes by name or power"))
        .performTextInput("Spider")
    composeTestRule.onNode(hasText("Spider-Man")).assertIsDisplayed()
    composeTestRule.onNode(hasText("Iron Man")).assertDoesNotExist()
}

// ── Test 2: Search by power keyword filters correctly ────────────
@Test
fun search_byPowerKeyword_filtersCorrectly() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    composeTestRule
        .onNode(hasText("Search heroes by name or power"))
        .performTextInput("flight")
    // Should show Iron Man, Wonder Woman, Superman, Thor
    composeTestRule.onNode(hasText("Iron Man")).assertIsDisplayed()
    composeTestRule.onNode(hasText("Batman")).assertDoesNotExist()  // no flight kw
}

// ── Test 3: Favourite toggle updates UI immediately ──────────────
@Test
fun favouriteToggle_updatesIconAndStateCorrectly() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    composeTestRule
        .onNode(hasContentDescription("Add Iron Man to favourites"))
        .performClick()
```

```
    composeTestRule
        .onNode(hasContentDescription("Remove Iron Man from favourites"))
        .assertIsDisplayed()
}

// ── Test 4: Favourite filter shows only favourited heroes ──────────
@Test
fun favouriteFilter_showsOnlyFavouritedHeroes() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    composeTestRule.onNode(hasText("Favourites")).performClick()
    // Spider-Man and Wonder Woman are pre-favourited
    composeTestRule.onNode(hasText("Spider-Man")).assertIsDisplayed()
    composeTestRule.onNode(hasText("Iron Man")).assertDoesNotExist()
}

// ── Test 5: Detail screen shows correct hero data ──────────────────
@Test
fun detailScreen_showsCorrectHeroData() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    composeTestRule.onNode(hasText("Spider-Man")).performClick()
    composeTestRule.onNode(hasText("Peter Parker")).assertIsDisplayed()
    composeTestRule.onNode(hasText("Avengers")).assertIsDisplayed()
}

// ── Test 6: All hero images have contentDescription ────────────────
@Test
fun heroImages_allHaveNonEmptyContentDescription() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    // Verify at least Spider-Man's image has correct contentDescription
    composeTestRule
        .onNode(hasContentDescription("Spider-Man photo"))
        .assertIsDisplayed()
}

// ── Test 7: State preserved across orientation change ──────────────
@Test
fun listState_preservedAcrossOrientationChange() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    composeTestRule
        .onNode(hasText("Search heroes by name or power"))
        .performTextInput("Thor")
    // Simulate orientation change by recreating Activity
    composeTestRule.activityRule.scenario.recreate()
    // ViewModel survives — searchQuery state preserved
    composeTestRule.onNode(hasText("Thor")).assertIsDisplayed()
}

// ── Test 8: Back navigation returns to list screen ─────────────────
@Test
fun backNavigation_returnsToListScreen() {
    composeTestRule.setContent { SuperheroesApp(testViewModel) }
    composeTestRule.onNode(hasText("Spider-Man")).performClick()
    composeTestRule
        .onNode(hasContentDescription("Navigate back to heroes list"))
        .performClick()
```

```
    composeTestRule.onNode(hasText("Heroes Directory")).assertIsDisplayed()
}
```

6. Performance & Scalability Considerations

Concern	Strategy	Implementation Detail
50+ heroes in LazyColumn	Lazy composition — only visible items composed	LazyColumn with key = { hero.id }; off-screen items disposed automatically
Hero image memory	Constrain decoded size to display dp × density	Coil .size(400, 400) on ImageRequest; memory cache capped at 20% app memory
Image disk caching	Persist decoded images across sessions	Coil DiskCache at 100MB; memoryCacheKey = hero.imageUrl
Search query debouncing	Avoid refiltering on every keystroke	searchQuery StateFlow + debounce(300ms) in ViewModel before triggering combine()
Large dataset (200+ heroes)	Jetpack Paging 3 for paginated backend	PagingSource + LazyPagingItems; load 20 heroes per page
Image placeholder	Show shimmer while loading, error on failure	AsyncImage: placeholder = shimmer drawable; error = R.drawable.ic_broken_hero
Combine() overhead	4-flow combine recomputes on any change	Only displayedHeroes recomputes; allHeroes StateFlow mutated atomically via update {}
Image compression	Upload hero images at 400×400px max	Backend CDN serves WebP at 400px; Coil decodes at display size, not source size

7. Reflection

Designing the Superheroes Directory was significantly more complex than earlier list features for three compounding reasons. First, the data model grew in richness: the Superhero class with powerKeywords (a List<String>) required a multi-field search that checks the hero name, real name, and each keyword — a more sophisticated filter than the single-field search in earlier sessions. Second, the combination of three simultaneous filter dimensions (search query, favourite toggle, universe selector) using combine() on four StateFlows meant that any change to any filter instantly recomputes the entire displayed list, requiring careful thought about derived state to avoid redundant recomputations and jank in the search experience. Third, every interactive element in a two-screen app (list screen + detail screen) needed its own accessibility semantic — content descriptions, state descriptions, roles, and heading semantics — multiplying the number of things to verify against the audit checklist. The eight automated tests were not just verification tools but architectural guides: writing testSearch_byPowerKeyword() before implementing the filter logic forced a clear contract ('filter

must search powerKeywords list, not just powerSummary string') that shaped the data model design. Accessibility testing similarly shaped the semantic API choices — the decision to set contentDescription on the IconButton rather than the Icon inside it was discovered while writing the favouriteToggle_updatesIconAndStateCorrectly test, because TalkBack reads the IconButton's semantics node, not the inner Icon's. This reinforced the principle that test-first thinking — even at the planning stage — produces cleaner, more intentional implementation decisions.

TASK 3 — 30-Day Compose Roadmap & Realization Plan

1. 30-Day Project Vision

App: Compose Heroes Hub

Compose Heroes Hub is a fully polished, production-grade Android app showcasing 50+ superheroes from Marvel and DC, built entirely in Jetpack Compose over 30 days. The app integrates everything learned in Sessions 10–13: a Material 3 theme system with light/dark/dynamic colour support, a responsive multi-column list with Coil image loading, a real-time search and filter engine using Kotlin StateFlow, animated card interactions (expand, favourite, list entry/exit), a navigation-driven detail screen, comprehensive accessibility with TalkBack support, automated UI tests covering all key scenarios, and performance profiling to ensure 60fps on mid-range devices. By Day 30, the app will be presentable as a professional portfolio piece demonstrating full-stack Compose competency.

2. Milestone Planning — Five 6-Day Milestones

Milestone	Days	Goal	Tasks	Expected Outcome	Success Criteria
M1: Foundation	1–6	Set up project with correct architecture and full theme system	1. Create Android project (Compose BOM, Coil, Navigation, ViewModel). 2. Implement Color.kt, Typography.kt, Shapes.kt, Theme.kt with light/dark/dynamic colour. 3. Define Superhero data class and full 50-hero dataset. 4. Confirm MainActivity wraps app in WanderAppTheme.	App compiles. Theme switch between light/dark works. Dataset of 50 heroes populated with URLs and fallback drawables.	App builds and runs. TopAppBar renders with brand colours. isSystemInDarkTheme() flips theme correctly. No hard-coded colours.
M2: List UI	7–12	Build complete Heroes List screen with image loading	1. Implement HeroCard composable (image, hero name, real name, power snippet, universe chip, favourite icon). 2. LazyColumn with key = { hero.id }. 3. Coil AsyncImage with placeholder/error/fallback. 4. Responsive layout: LazyVerticalGrid(2) on landscape. 5. All theme tokens applied (shapes.medium, typography.titleLarge, colorScheme.surface).	All 50 heroes displayed in scrollable list. Images load from URLs with shimmer placeholder. Dark mode renders correctly. 2-column grid on tablet.	Scroll 50 items without jank. Coil loads and caches images. Layout inspector shows no hard-coded colours. Dark mode visual test passes.

Milestone	Days	Goal	Tasks	Expected Outcome	Success Criteria
M3: Search, Filter & Favourites	13–18	Add real-time search, universe filter, favourite filter, and favourite toggle	1. ViewModel with combine() for displayedHeroes. 2. searchQuery StateFlow with 300ms debounce. 3. Search bar with OutlinedTextField (accessible label). 4. Universe filter chips (All/Marvel/DC). 5. Favourites filter chip. 6. toggleFavourite() with StateFlow.update {}. 7. AnimatedVisibility for hero exit when unfavourited under filter.	Search by name filters list in real time. Searching 'flight' shows only flight-capable heroes. Favourite toggle persists across scroll. Favourite filter shows only favourites.	Test 1–4 from test suite PASS. Searching 'Spider' shows Spider-Man only. 300ms debounce confirmed in profiler. State preserved on rotation.
M4: Detail Screen, Animations & State	19–24	Navigation to detail screen + coordinated animations on list and detail	1. NavHost: heroes list → hero detail. 2. Hero detail screen (full image, power summary, team, favourite button). 3. updateTransition on HeroCard (bg colour, elevation, content alpha). 4. keyframes for favourite icon. 5. AnimatedVisibility enter/exit for list items. 6. rememberAnimationSpec() helper for Reduce Motion. 7. Shared state: toggle on detail also updates list.	Tapping a card navigates to detail. Tapping Back returns to list. Card expand animation is smooth (spring). Favourite on detail reflects on list. Reduce Motion path tested.	Navigation round-trip test PASSES. updateTransition profiled at 60fps. Detail screen shows correct hero data (Test 5 PASSES). Animation disabled when animatorDurationScale=0.
M5: Accessibility, Tests, Performance & Presentation	25–30	Full accessibility audit, complete test suite, performance profiling, and final presentation	1. Perform full accessibility audit using audit checklist from Task 1. 2. Fix all High-priority issues (contentDescription, Reduce Motion, touch targets, semantics). 3. Write all 8 automated tests (Task 2 test suite). 4. Run Android Studio Profiler — verify 60fps on Pixel 4a equivalent. 5. Run AccessibilityChecks Espresso rule. 6. Prepare 4-slide presentation deck. 7. Demo to mentor/team.	All 8 automated tests PASS. TalkBack reads all elements correctly. No High-priority audit issues outstanding. App demo-ready on Pixel 6 emulator.	All tests green. Profiler shows < 16ms per frame on expand animation. Zero WCAG AA failures in AccessibilityChecks. Presentation delivered.

2.1 Daily Breakdown for Milestone 1 (example)

Day	Task	Deliverable
Day 1	Create project: Compose BOM, Coil, Navigation, ViewModel Compose, Kotlin Coroutines dependencies in build.gradle.kts	Project builds; no dependency conflicts
Day 2	Color.kt: define all 12 light + 12 dark token values; WanderLightColorScheme + WanderDarkColorScheme	Both schemes declared; no compile errors
Day 3	Typography.kt: Montserrat + Roboto font families; 8 TextStyle tokens	WanderTypography declared; custom fonts load
Day 4	Shapes.kt + Theme.kt: WanderShapes; WanderAppTheme with dynamic colour logic	Theme switch works in Preview
Day 5	Superhero data class + first 25 hero entries in dataset	Data class compiles; 25 heroes in list
Day 6	Remaining 25 hero entries; ViewModel skeleton with _allHeroes StateFlow; MainAcivity sets content	50 heroes in ViewModel; app launches with themed background

3. Resource & Tooling Plan

3.1 Tools & Libraries

Tool / Library	Version	Purpose
Android Studio Hedgehog+	Latest stable	IDE; Layout Inspector; Profiler; Compose Preview
Kotlin	2.0.x	Language; coroutines; Flow
Jetpack Compose BOM	2024.09.00	Core Compose, Material 3, Animation, Navigation
coil-compose	2.6.0	AsyncImage; disk cache; memory cache
androidx.navigation:navigation-compose	2.8.x	NavHost; NavController; type-safe routes
lifecycle-viewmodel-compose	2.8.x	ViewModel + collectAsStateWithLifecycle
kotlinx-coroutines-android	1.8.x	StateFlow; combine; debounce
androidx.compose.ui:ui-test-junit4	BOM	Compose UI testing; composeTestRule

Tool / Library	Version	Purpose
AccessibilityChecks (Espresso)	4.0.0	WCAG AA automated contrast + target checks
GitHub Actions	Latest	CI: run tests on every push; fail-fast on test failure
GitHub	N/A	Version control; branch per milestone; PR for review
SuperheroAPI.com or JSON local file	N/A	Source of 50 hero data entries and image URLs

3.2 Learning Resources (by Milestone)

Milestone	Primary Resources
M1 (Theming)	developer.android.com/develop/ui/compose/designsystems/material3 | Material 3 Theme Builder tool | Android Developers YouTube: 'Material You in Compose'
M2 (List UI)	Coil docs: coil-kt.github.io/coil | Compose LazyColumn codelab (developer.android.com) | Philip Lackner YouTube: 'Compose Lists'
M3 (Search/Filter)	Kotlin Flow docs: kotlinlang.org/docs/flow | Medium: 'Debouncing with StateFlow' | developer.android.com/kotlin/flow/stateflow-and-sharedflow
M4 (Animation)	developer.android.com/develop/ui/compose/animation | YouTube: 'Compose Animation APIs' (Google I/O) | Session 12 notes from MindMatrix
M5 (Accessibility)	developer.android.com/develop/ui/compose/accessibility | WCAG 2.1 Quick Reference | Session 13 audit checklist from MindMatrix

4. Risk & Mitigation Table

#	Risk	Likelihood	Impact	Mitigation Strategy	Owner
R1	Hero image URLs unavailable or rate-limited (SuperheroAPI)	Medium	High — blank images throughout app	Use local drawable fallbacks for all 50 heroes from day 1; only add URLs when confirmed working; store images in res/drawable as backup	Day 1
R2	combine() on 4 StateFlows causes excessive recomposition /	Medium	Medium — poor search UX on older devices	Add 300ms debounce to searchQuery before it enters combine(); profile with CPU	Day 13

#	Risk	Likelihood	Impact	Mitigation Strategy	Owner
	jank during search			Profiler on Day 16 after filter is complete	
R3	Animation causing frame drops on low-end device (Pixel 2 equivalent)	Medium	Medium — janky feel on target market devices	Pre-test updateTransition on API 26 emulator on Day 20; use graphicsLayer for all scale/alpha; limit simultaneous animations to 1 per card	Day 19
R4	Accessibility test failures block CI pipeline on Day 25+	Medium	High — late-stage rework required	Run TalkBack manual audit and at least 3 automated tests from Day 7 onward, not just Day 25; fix issues in same milestone they are found	Day 7+
R5	ViewModel combine() state lost on process death (app killed)	Low	Medium — user loses search query and favourites on restart	For v1, in-memory state is acceptable; document as known limitation; plan Room database persistence for v2 post-30-day roadmap	Day 30 doc
R6	Dataset curation takes longer than expected (50 hero entries)	High	Low — only cosmetic/data delay	Use CSV import script to generate Kotlin data file from spreadsheet; reduce to 30 heroes if 50 is not achievable by Day 6	Day 1–6
R7	GitHub Actions CI fails due to Compose test emulator setup complexity	Medium	Low — slows CI but doesn't block local dev	Use GitHub-hosted macOS runner with hardware acceleration; use ftl-android-emulator alternative; have local test run as fallback	Day 7

5. Reflection & Commitment

What excites me most about this 30-day project is that it represents the first time all of the individual skills learned across Sessions 10–13 will be combined into a single, coherent, production-grade product. In earlier sessions, each concept — theming, lambdas, LazyColumn, animation, accessibility — was explored in relative isolation on a focused feature. The Compose Heroes Hub will demand that all of these layers work together simultaneously, and the integration challenges — like ensuring theme colour tokens are read inside animation lambdas, or that ViewModel StateFlow correctly drives both the search filter and the favourite toggle — are exactly the kinds of problems that do not appear in isolated

exercises but are central to real-world Android development. The project will deepen understanding of Kotlin Flow's combine() operator in a meaningful way, because the 3-filter search system genuinely requires it rather than serving as a contrived example. Progress will be measured concretely at each milestone boundary using the defined success criteria — not just 'I wrote some code' but 'all tests for this milestone pass and the profiler shows no regressions.' Accountability will come from daily commits to GitHub (each commit message describes what was completed that day), a weekly 30-minute review session where the milestone deliverable is demoed against its success criteria, and a peer review of the codebase at the end of each milestone. If a milestone is not met on schedule, one day of buffer from the following milestone will be used — never skipping the accessibility or testing work to make up time, as those are non-negotiable quality gates. The 30-day project is not just a coding exercise — it is proof that the skills are integrated and transferable.

Session 13 — Learning Summary

Consolidated key learnings from all three tasks in Session 13 — Accessibility Audit, Superheroes Feature Planning, and 30-Day Compose Roadmap.

Key Concepts Mastered

Concept	What I Learned	Where Applied
WCAG AA contrast ratios	4.5:1 minimum for normal text; 3:1 for large text; 7:1 for AAA	Task 1 — audit items 9–13
contentDescription patterns	Set on interactive element (IconButton), not inner Icon; update dynamically with state	Task 1, 2 — favourite icon semantics
Modifier.semantics block	stateDescription, role, heading, contentDescription all set here	Task 1, 2 — card expand, filter chips
animatorDurationScale = 0	System setting for Reduce Motion; check via Settings.Global; use snap() when 0	Task 1, 2 — rememberAnimationSpec() helper
Compose UI Test semantics	hasContentDescription, hasStateDescription, hasText, assertIsDisplayed	Task 1, 2 — 6 and 8 test cases respectively
Superhero data class design	powerKeywords: List<String> enables multi-field search; alias: String? for nullable codename	Task 2 — data model
combine() for multi-filter	4 StateFlows merged into one displayedHeroes; reactive to any single change	Task 2 — ViewModel search + filter logic
searchQuery debounce	debounce(300ms) on searchQuery StateFlow prevents refilter on every keystroke	Task 2 — performance
minimumInteractiveComponentSize()	Ensures 48dp touch target even if content is smaller	Task 2 — favourite IconButton

Concept	What I Learned	Where Applied
30-day milestone planning	5 × 6-day milestones; each has goal, tasks, outcome, success criteria	Task 3 — full roadmap
Risk & Mitigation table	Identify risks by likelihood × impact; assign mitigation strategy and owner	Task 3 — 7 risks identified
GitHub Actions CI	Run Compose UI tests on every push; emulator setup; fail-fast on test failure	Task 3 — tooling plan

Best Practices Identified

- Set contentDescription on the interactive container (IconButton, Button) — not on the icon or image inside it — so TalkBack announces the action, not the visual element.
- Use Modifier.minimumInteractiveComponentSize() rather than Modifier.size(48.dp) for touch targets — it enforces the minimum without overriding intentional smaller sizes.
- Always check animatorDurationScale at runtime using rememberAnimationSpec() — never assume the user wants motion.
- Use combine() with 3+ StateFlows for multi-dimensional filters — it guarantees the displayed list is always derived from the current combination of all filters simultaneously.
- Write automated tests alongside feature design, not after — tests shape cleaner API decisions (setting contentDescription on IconButton vs Icon was discovered writing tests).
- Add searchQuery debounce (300ms) before it enters combine() — avoids refiltering on every keystroke, which would be visible as list flicker at 60fps.
- Run TalkBack audit from Day 7, not Day 25 — late-stage accessibility fixes require architectural rework; early fixes are one-line modifier additions.
- Define milestone success criteria as test outcomes — 'tests green' is a more reliable completion gate than 'feature looks done'.

Student Declaration

I, Sudarshan G, confirm that all work submitted in this assessment document is my own. The accessibility audit, improvement plan, test cases, Superheroes feature plan, and 30-Day roadmap represent my own understanding of the concepts covered in Session 13 of the MindMatrix Android Development programme.

Student: Sudarshan G

Session: Session 13 — Accessibility, Superheroes Feature & 30-Day Roadmap

Programme: Android App Development with Kotlin & Jetpack Compose